Assignment 2: Data Preprocessing and Cleaning

- Choose a large dataset from a public repository (e.g., Kaggle).
- Apply data cleaning techniques such as handling missing values, outliers, and noise.
- Use Apache Spark or a similar framework to parallelize the data processing tasks.
- Split the dataset into training and testing sets for future analysis.

For **Assignment 2: Data Preprocessing and Cleaning**, do this practical using **Apache Spark (PySpark)** because it directly demonstrates how Big Data preprocessing is performed in a distributed environment.

Below is a **complete, practical guide**, including dataset selection, installation, code, explanation, expected output, and viva questions.

Assignment 2: Data Preprocessing and Cleaning Using Apache Spark

1. Aim

To perform data preprocessing and cleaning on a large dataset using Apache Spark, including handling missing values, outliers, and noisy data, and to split the cleaned dataset into training and testing sets.

2. Objectives

After completing this practical, students should be able to:

1. Obtain a large dataset from a public repository such as Kaggle.
2. Load a CSV dataset into Apache Spark.
3. Understand the structure and quality of the dataset.
4. Detect missing/null values.
5. Handle missing values using appropriate techniques.
6. Detect and handle outliers.
7. Identify and reduce noisy/invalid data.
8. Remove duplicate records.
9. Perform data type conversion.
10. Use Spark transformations and actions for preprocessing.
11. Divide the dataset into training and testing datasets.
12. Save the cleaned dataset for future analysis.

3. Software Requirements

Recommended setup

For beginners, **Google Colab + PySpark** is often easier than configuring a complete Hadoop/Spark cluster on Windows.

Requirement	Software
Operating System	Windows / Linux
Programming Language	Python
Big Data Framework	Apache Spark
API	PySpark
IDE	VS Code / Jupyter Notebook
Dataset	Kaggle / UCI / other public repository
Java	JDK 8/11/17 depending on Spark version

4. Dataset Selection

Students can select a sufficiently large public dataset.

Recommended datasets

Option 1 – NYC Taxi Trip Dataset
Contains information such as:

- Pickup date/time
- Drop-off date/time
- Passenger count
- Trip distance
- Fare amount
- Payment type
- Location information

This is particularly suitable for demonstrating Big Data processing because taxi datasets can contain millions of records.

Other possible datasets

- Airline Flight Dataset
- Amazon Customer Reviews
- COVID-19 Dataset
- E-commerce transactions
- MovieLens dataset
- Road accident dataset
- Financial transaction dataset

For this assignment **Recommendation**

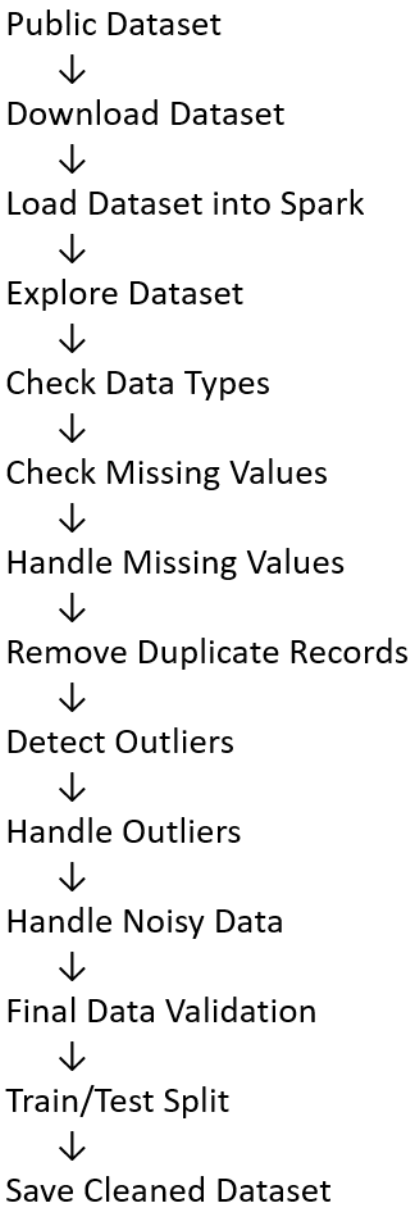
NYC Taxi Trip Dataset

because it provides opportunities to demonstrate:

- Missing values
- Duplicate records
- Invalid values
- Outliers
- Numerical preprocessing
- Date/time preprocessing
- Train/test splitting

5. Practical Workflow

The complete workflow is:



6. Step 1 – Install PySpark
Open Command Prompt or VS Code terminal.

</> Bash
```
pip install pyspark
```

Verify installation:
</> Bash

```
python -c "import pyspark; print(pyspark.__version__)"
```
If Python is configured through the Windows launcher, you can also try:
</> Bash

```
py -m pip install pyspark
```

7. Step 2 – Import Required Libraries

Create a file:

`assignment2_preprocessing.py`

Add:

</>python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *
```

#After Run
Run started
Initializing environment
Installing packages
Running code
ModuleNotFoundError: No module named 'pyspark'
Run completed in 1378.3999999910593ms

8. Step 3 – Create Spark Session

</>python

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("BigDataDataPreprocessing") \
    .master("local[*]") \
    .getOrCreate()

print("Spark Session Created Successfully")
```

#After Run
Run started
Initializing environment
Installing packages
Running code
ModuleNotFoundError: No module named 'pyspark'
Run completed in 1651.9000000059605ms

Explanation
</>python

```
.master("local[*]")
```

means Spark can use the available CPU cores on the local machine.
In a real cluster, Spark can distribute processing across multiple machines.

#After Run
Run started
Initializing environment
Installing packages
Running code
SyntaxError: invalid syntax (<exec>, line 1)
Run completed in 1220.5ms

9. Step 4 – Load the Dataset

Suppose the downloaded file is:

`taxi_data.csv`

Place it in the same folder as your Python program.

Use:

</>python

```
df = spark.read.csv(
    "taxi_data.csv",
    header=True,
    inferSchema=True
)
```

#After Run
Run started
Initializing environment
Installing packages
Running code
NameError: name 'spark' is not defined
Run completed in 1005ms

Explanation

Parameter	Meaning
`header=True`	First row contains column names
`inferSchema=True`	Spark automatically determines data types
`read.csv()`	Reads CSV data into a Spark DataFrame

10. Step 5 – Display Dataset

Display first 10 records:

```
</>python
df.show(10)
```

#After Run

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 1021.2000000029802ms

Display column names:

```
</>python
print(df.columns)
```

#AfterRun

Run started
Initializing environment
Installing packages
Running code
NameError: name 'df' is not defined
Run completed in 1072.4000000059605ms

Display number of records:

```
</>python
print("Number of records:", df.count())
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df' is not defined
Run completed in 1002.7999999970198ms

Display number of columns:

```
</>python

print("Number of columns:",
len(df.columns))
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df' is not defined
Run completed in 1023.7000000029802ms

11. Step 6 – Understand Data Types

Use:

</>Python

```
df.printSchema()
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 1084.4000000059605ms

Example:

```
root
 |-- passenger_count: integer
 |-- trip_distance: double
 |-- fare_amount: double
 |-- payment_type: string
```

Understanding data types is important because different preprocessing techniques are applied to numerical and categorical columns.

12. Step 7 – Generate Basic Statistics

Use:

</>Python

```
df.describe().show()
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 1092.099999940395ms

This gives:

- Count
- Mean
- Standard deviation
- Minimum
- Maximum

Example:

```
+-------+---------------+-------------+
|summary|trip_distance  |fare_amount  |
+-------+---------------+-------------+
|count  |...            |...          |
|mean   |...            |...          |
|stddev |...            |...          |
|min    |...            |...          |
|max    |...            |...          |
+-------+---------------+-------------+
```

This is useful for identifying potential outliers.

13. Step 8 – Identify Missing Values

Missing values can occur because:

- Data was not recorded.
- Sensors failed.
- Users did not provide information.
- Data collection was interrupted.
- Different sources use different formats.

Count null values in every column

</>Python

```
df.select([
    count(when(col(c).isNull(),
c)).alias(c)
    for c in df.columns
]).show()
```

This tells us how many null values exist in each column.

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 961.799999970198ms

14. Step 9 – Handling Missing Values

There are several techniques.

Method 1 – Remove records

If only a small number of records contain missing values:

</>Python

```
df_clean = df.dropna()
```

However, this should **not** automatically be used when many records contain missing values.

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 1070.4000000059605ms

Method 2 – Fill numerical values

For example, replace missing passenger counts with the median or another appropriate value.

A simple approach is:

</>Python

```
df_clean = df.fillna({
    "passenger_count": 1
})
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 1104.799999970198ms

Method 3 – Fill categorical values

For example:

```
</>Python
df_clean = df.fillna({
    "payment_type": "Unknown"
})
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df' is not defined

Run completed in 1041.9000000059605ms

15. Better Approach – Median Imputation

For numerical Big Data, median is often preferable when extreme values exist.

Example:

```
</>Python

median_value = df.approxQuantile(
    "trip_distance",
    [0.5],
    0.01
)[0]

print("Median:", median_value)
```

```
#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df' is not defined
Run completed in 1049.9000000059605ms
```

Then:

```
</>Python

df_clean = df.fillna({
    "trip_distance": median_value
})
```

```
#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df' is not defined
Run completed in 1054.799999970198ms
```

Why median?

Suppose:

```
Trip distance:
2
3
4
5
1000
```

Mean is strongly affected by `1000`, while median remains representative.

16. Step 10 – Remove Duplicate Records

Duplicate records can distort analysis.

Check:

```
</>Python

print("Before removing duplicates:",
df_clean.count())
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df_clean' is not defined

Run completed in 1115.099999940395ms

Remove duplicates:

```
</>Python

df_clean = df_clean.dropDuplicates()
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df_clean' is not defined
Run completed in 996.7999999970198ms

Check again:
</>Python

```
print("After removing duplicates:",
df_clean.count())
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df_clean' is not defined
Run completed in 1033.2999999970198ms

17. Step 11 – Detect Invalid Values

Data can contain values that are technically not missing but are logically incorrect.

For example:

```
passenger_count = -2
trip_distance = -10
fare_amount = -50
```

These are invalid.

Check passenger count:

```
</>Python
df_clean.filter(
    col("passenger_count") <= 0
).show()
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df_clean' is not defined

Run completed in 1038ms

Check trip distance:

```
</>Python

df_clean.filter(
    col("trip_distance") <= 0
).show()
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df_clean' is not defined
Run completed in 1036.1000000089407ms

Check fare:

```
</>Python

df_clean.filter(
    col("fare_amount") <= 0
).show()
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df_clean' is not defined

Run completed in 1115.7000000029802

18. Step 12 – Remove Invalid Records

For example:

</>Python

```
df_clean = df_clean.filter(
    (col("passenger_count") > 0) &
    (col("trip_distance") > 0) &
    (col("fare_amount") > 0)
)
```

Now the dataset contains only logically valid records.

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df_clean' is not defined

Run completed in 940ms

19. Step 13 – Detect Outliers

An **outlier** is a data value that is significantly different from most other values.

Example:

```
2
3
4
5
6
7
500
```

Here, `500` may be an outlier.

Common techniques:

1. Z-score
2. IQR
3. Standard deviation
4. Domain-specific thresholds

For this practical, use the **IQR method**.

20. IQR Method

IQR stands for:

Interquartile Range

Formula:

$$IQR = Q3 - Q1$$

where:

- Q1 = First Quartile
- Q3 = Third Quartile

Lower bound:

$$Lower = Q1 - 1.5(IQR)$$

Upper bound:

$$Upper = Q3 + 1.5(IQR)$$

Values outside these limits can be treated as potential outliers.

21. Implement IQR in PySpark

Suppose we want to examine:

`trip_distance`

Calculate Q1 and Q3:

</>python

```
quantiles = df_clean.approxQuantile(
    "trip_distance",
    [0.25, 0.75],
    0.01
)

Q1 = quantiles[0]
Q3 = quantiles[1]

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df_clean' is not defined
Run completed in 1010.1999999880791ms

22. Identify Outliers

</>python

```
outliers = df_clean.filter(
    (col("trip_distance") < lower_bound) |
    (col("trip_distance") > upper_bound)
)

print("Number of outliers:",
outliers.count())

outliers.show(10)
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df_clean' is not defined
Run completed in 1670.699999988079ms

23. Handling Outliers

There are several options.

Option 1 – Remove outliers

</>python

```
df_clean = df_clean.filter(
    (col("trip_distance") >= lower_bound) &
    (col("trip_distance") <= upper_bound)
)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df_clean' is not defined

Run completed in 1130.099999940395ms

Option 2 – Cap the values

Instead of deleting records, values can be capped.

Conceptually:

```
If value < lower bound → replace with lower bound

If value > upper bound → replace with upper bound
```

In a practical assignment, students can demonstrate either approach and explain why they selected it.

24. Step 14 – Handling Noise

Noise means unwanted random variation, errors, inconsistencies, or incorrect observations in the data.

Examples:

```
Payment:
cash
Cash
CASH
unknown
?
```

These represent inconsistent categories.

25. Standardize Categorical Data

Suppose a column contains:

```
cash
Cash
CASH
```

We can convert it to a common format.

</>Python

```
df_clean = df_clean.withColumn(
    "payment_type",
    lower(trim(col("payment_type")))
)
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df_clean' is not defined
Run completed in 1006ms

Now:

```
cash
Cash
CASH
```

becomes:

```
cash
cash
cash
```

26. Handle Unknown Values

For example:

</>Python

```
df_clean = df_clean.withColumn(
    "payment_type",
    when(
        col("payment_type").isNull(),
        "unknown"
    ).otherwise(col("payment_type"))
)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df_clean' is not defined

Run completed in 1036.199999988079ms

We can also replace certain unwanted symbols:

```
</>Python
df_clean = df_clean.replace(
    ["?", "NA", "N/A", ""],
    "unknown"
)
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df_clean' is not defined
Run completed in 1032.099999940395ms

27. Step 15 – Data Type Conversion

Sometimes data is read as the wrong type.

For example, a numerical column may be stored as a string.

Use:

</>Python

```
df_clean = df_clean.withColumn(
    "fare_amount",
    col("fare_amount").cast("double")
)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df_clean' is not defined

Run completed in 1049.599999940395ms

Similarly:

</>Python

```
df_clean = df_clean.withColumn(
    "passenger_count",
    col("passenger_count").cast("integer")
)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df_clean' is not defined

Run completed in 1186.5ms

28. Step 16 – Final Data Quality Check

After cleaning:

</>Python

```
df_clean.printSchema()
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df_clean' is not defined
Run completed in 1018.9000000059605ms

Check missing values again:
</>Python

```
df_clean.select([
    count(when(col(c).isNull(),
c)).alias(c)
    for c in df_clean.columns
]).show()
```

#AfterRun
Run
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df_clean' is not defined
Run completed in 1048.099999940395ms

Check records:
</>Python

```
print("Final number of records:",
df_clean.count())
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df_clean' is not defined
Run completed in 1057ms

Display cleaned data:

</>Python

```
df_clean.show(10)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df_clean' is not defined

Run completed in 1042ms

29. Step 17 – Train/Test Split

The cleaned dataset can now be divided into:

- **Training dataset**
- **Testing dataset**

A common split is:

```
80% Training
20% Testing
```

Use:

</>Python

```
train_df, test_df = df_clean.randomSplit(
    [0.8, 0.2],
    seed=42
)
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df_clean' is not defined

Run completed in 1036.5ms

Check:
</>Python

```
print("Training records:",
train_df.count())
print("Testing records:", test_df.count())
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'train_df' is not defined
Run completed in 1066.4000000059605ms

30. Why Train/Test Split?

The training dataset is used to:

- Build a machine learning model.
- Learn patterns.
- Estimate model parameters.

The testing dataset is used to:

- Evaluate the model.
- Check how well it performs on unseen data.

Example

If there are:

```
1,000,000 records
```

approximately:

```
Training → 800,000
Testing  → 200,000
```

31. Step 18 – Save Cleaned Data

Save the cleaned dataset as CSV:

</>Python

```
df_clean.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("cleaned_data")
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'df_clean' is not defined

Run completed in 1061.7000000178814ms

Save training data:
</>Python

```
train_df.write \
     .mode("overwrite") \
     .option("header", True) \
     .csv("training_data")
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'train_df' is not defined
Run completed in 1059.2000000178814ms

Save testing data:
</>Python

```
test_df.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("testing_data")
```

#AfterRun

Run started
Initializing environment
Installing packages
Running code
NameError: name 'test_df' is not defined
Run completed in 1023.4000000059605ms

32. Better Big Data Format – Parquet

Spark works particularly well with **Parquet**, a columnar storage format.

Instead of CSV:

</>Python

```
df_clean.write \
    .mode("overwrite") \
    .parquet("cleaned_data.parquet")
```

#AfterRun

Run
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df_clean' is not defined
Run completed in 1009.9000000059605ms

Training:
</>Python

```
train_df.write \
     .mode("overwrite") \
     .parquet("training_data.parquet")
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'train_df' is not defined
Run completed in 1014.899999761581ms

Testing:
</>Python

```
test_df.write \
    .mode("overwrite") \
    .parquet("testing_data.parquet")
```

For a **Big Data practical**, I recommend asking students to demonstrate Parquet because it connects the preprocessing task to practical Big Data storage.

#AfterRun

Run started
Initializing environment
Installing packages
Running code
NameError: name 'test_df' is not defined
Run completed in 987.3000000119209ms

For a **Big Data practical**, demonstrate Parquet because it connects the preprocessing task to practical Big Data storage.

33. Complete Practical Program

Students can combine the steps into one program.

</>Python

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *

# ------------------------------------------------
# 1. Create Spark Session
# ------------------------------------------------

spark = SparkSession.builder \
    .appName("DataPreprocessing") \
    .master("local[*]") \
    .getOrCreate()

print("Spark started successfully")
```

```
# ------------------------------------------------
# 2. Load Dataset
# ------------------------------------------------

df = spark.read.csv(
    "taxi_data.csv",
    header=True,
    inferSchema=True
)

print("Original Dataset")
df.show(5)



# ------------------------------------------------
# 3. Dataset Information
# ------------------------------------------------

print("Number of records:", df.count())
print("Number of columns:", len(df.columns))

df.printSchema()
```

```
# ------------------------------------------------
# 4. Statistical Summary
# ------------------------------------------------

df.describe().show()


# ------------------------------------------------
# 5. Missing Values
# ------------------------------------------------

print("Missing values:")

df.select([
    count(when(col(c).isNull(), c)).alias(c)
    for c in df.columns
]).show()
```

```
# ------------------------------------------------
# 6. Handle Missing Values
# ------------------------------------------------

df_clean = df.fillna({
    "payment_type": "unknown",
    "passenger_count": 1
})



# ------------------------------------------------
# 7. Remove Duplicates
# ------------------------------------------------

df_clean = df_clean.dropDuplicates()
```

```
# ------------------------------------------------
# 8. Remove Invalid Values
# ------------------------------------------------

df_clean = df_clean.filter(
    (col("passenger_count") > 0) &
    (col("trip_distance") > 0) &
    (col("fare_amount") > 0)
)



# ------------------------------------------------
# 9. Standardize Text Data
# ------------------------------------------------

if "payment_type" in df_clean.columns:

    df_clean = df_clean.withColumn(
        "payment_type",
        lower(trim(col("payment_type")))
    )
```

```
# ------------------------------------------------
# 10. Detect Outliers Using IQR
# ------------------------------------------------

quantiles = df_clean.approxQuantile(
    "trip_distance",
    [0.25, 0.75],
    0.01
)

Q1 = quantiles[0]
Q3 = quantiles[1]

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
```

```
# ------------------------------------------------
# 11. Remove Outliers
# ------------------------------------------------

df_clean = df_clean.filter(
    (col("trip_distance") >= lower_bound) &
    (col("trip_distance") <= upper_bound)
)
```

```
# ------------------------------------------------
# 12. Final Dataset
# ------------------------------------------------

print("Cleaned Dataset")

df_clean.show(10)

print(
    "Final number of records:",
    df_clean.count()
)

# ------------------------------------------------
# 13. Train/Test Split
# ------------------------------------------------

train_df, test_df = df_clean.randomSplit(
    [0.8, 0.2],
    seed=42
)

print("Training records:", train_df.count())
print("Testing records:", test_df.count())
```

```
# ------------------------------------------------
# 14. Save Results
# ------------------------------------------------

train_df.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("training_data")

test_df.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv("testing_data")


# ------------------------------------------------
# 15. Stop Spark
# ------------------------------------------------

spark.stop()

print("Processing completed successfully")
```

#AfterRun
Run started
Initializing environment
Installing packages
Running code
ModuleNotFoundError: No module named 'pyspark'
Run completed in 1713.699999988079ms

34. Important Point for Students: Spark Parallel Processing

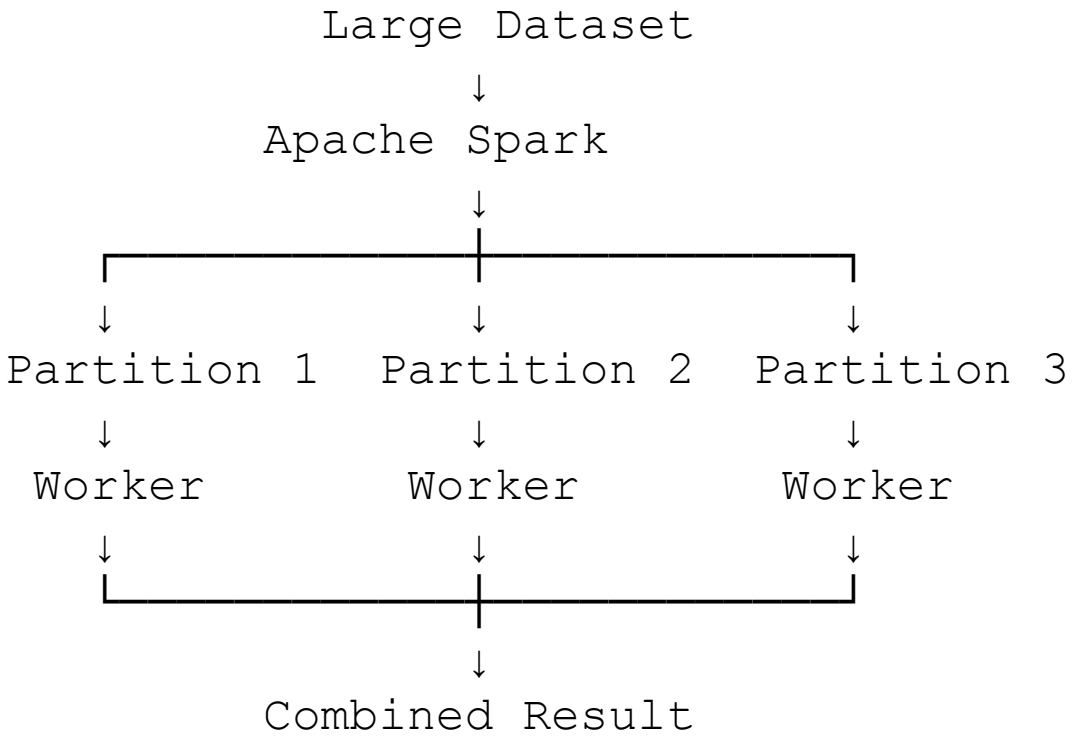
One of the main purposes of this assignment is **not simply cleaning a CSV using Python**.

Students should understand why Spark is being used.

Traditional Python/Pandas:

```
Large Dataset
     ↓
Single Machine
     ↓
Single/limited processing
     ↓
Memory limitation
```

Spark:


Large Dataset
Apache Spark
Partition 1
Partition 2
Partition 3
Worker
Worker
Worker
Combined Result

With:
</><Python
`.master("local[*]")`
Spark can use multiple cores on the student's computer.
In a true cluster environment, these partitions can be processed across multiple worker machines.

#AfterRun
Run started
Initializing environment
Installing packages
Running code
SyntaxError: invalid syntax (<exec>, line 1)
Run completed in 1141ms

35. Transformation vs Action

This is an important viva concept.

Transformations

Transformations create a new DataFrame/RDD and are generally evaluated lazily.

Examples:

</><Python

```
filter()
select()
withColumn()
drop()
dropDuplicates()
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

TypeError: filter expected 2 arguments, got 0

Run completed in 983.4000000059605ms

Actions

Actions actually trigger computation.

Examples:

</><Python

```
show()
count()
collect()
```

#AfterRun

Run started

Initializing environment

Installing packages

Running code

NameError: name 'show' is not defined

Run completed in 946.4000000059605ms

For example:
</><Python

```
df_clean = df.filter(col("fare_amount") > 0)
```

doesn't necessarily execute the complete computation immediately.

#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df' is not defined
Run completed in 1028.800000011921ms

But:
</><Python

```
df_clean.count()
```

triggers execution.
This is called **lazy evaluation**.

```
#AfterRun
Run started
Initializing environment
Installing packages
Running code
NameError: name 'df_clean' is not defined
Run completed in 1006ms
```

36. Expected Results Table

Students should maintain a table like this in their practical report:

Stage	Number of Records
Original dataset	XXXXX
After missing-value treatment	XXXXX
After duplicate removal	XXXXX
After invalid-value removal	XXXXX
After outlier removal	XXXXX
Final cleaned dataset	XXXXX
Training dataset	XXXXX
Testing dataset	XXXXX

They should use their **actual output values**, rather than copying numbers from another student's report.

37. Practical Report Format

Students can submit their practical in the following format.

Experiment No. 2

Title

Data Preprocessing and Cleaning Using Apache Spark

Aim

To preprocess and clean a large dataset using Apache Spark by handling missing values, outliers, duplicate records and noisy data, followed by training/testing data partitioning.

Objectives

List the objectives mentioned earlier.

Software Requirements

- Python
- PySpark
- Apache Spark
- VS Code/Jupyter
- Public dataset

Dataset Description

Include:

- Dataset name
- Source
- Number of records
- Number of attributes
- File format
- Description of important attributes

Theory

Explain:

- Data preprocessing
- Missing values
- Outliers
- Noise
- Duplicate records
- Data normalization/standardization
- Apache Spark
- DataFrame
- Parallel processing
- Train/test split

Algorithm

1. Start Spark
2. Load dataset
3. Explore dataset
4. Check missing values
5. Handle missing values
6. Remove duplicates
7. Detect invalid values
8. Detect outliers
9. Remove/handle outliers
10. Handle noisy data
11. Validate cleaned dataset
12. Split into training and testing datasets
13. Save datasets
14. Stop Spark

Program

Include the PySpark program.

Output

Students should include screenshots of:

1. Spark session
2. Dataset loading
3. Schema
4. Missing-value results
5. Statistical summary
6. Outlier detection
7. Cleaned dataset
8. Training/testing record counts

Result

The large dataset was successfully preprocessed and cleaned using Apache Spark. Missing values, duplicate records, invalid values, outliers and noisy data were handled, and the resulting dataset was successfully divided into training and testing datasets.

38. Viva Questions

Basic Questions

1. What is data preprocessing?

Data preprocessing is the process of cleaning and transforming raw data into a suitable form for analysis or machine learning.

2. Why is data cleaning necessary?

Because real-world datasets often contain missing, duplicate, inconsistent, noisy and incorrect data.

3. What is a missing value?

A missing value represents information that is unavailable or not recorded for an observation.

4. What is an outlier?

An outlier is an observation that differs significantly from the majority of observations.

5. What is noise?

Noise is unwanted variation, errors or random disturbances in data.

Spark Questions

6. What is Apache Spark?

Apache Spark is a distributed data-processing framework designed for fast processing of large datasets.

7. What is PySpark?

PySpark is the Python API for Apache Spark.

8. What is a Spark DataFrame?

A Spark DataFrame is a distributed collection of data organized into named columns.

9. What does `local[*]` mean?

It tells Spark to run locally using the available CPU cores.

10. What is lazy evaluation?

Spark delays execution of transformations until an action requires the result.

Data Cleaning Questions

11. How can missing values be handled?

Using:

- Removal
- Mean imputation
- Median imputation
- Mode imputation
- Constant-value replacement

12. What is IQR?

IQR is the difference between the third and first quartiles:

$$IQR = Q3 - Q1$$

13. What is the usual IQR outlier rule?

Values below:

$$Q1 - 1.5(IQR)$$

or above:

$$Q3 + 1.5(IQR)$$

are commonly treated as potential outliers.

14. Why remove duplicates?

Duplicates can cause incorrect counts and biased analysis.

15. Why split the data into training and testing sets?

To train a model on one portion and evaluate it on previously unseen data.

39. Suggested Student Deliverables

Ask each student/group to submit:

1. Dataset

`original_dataset.csv`

2. Python program

`assignment2_preprocessing.py`

3. Cleaned dataset

`cleaned_data/`

4. Training dataset

`training_data/`

5. Testing dataset

`testing_data/`

6. Practical report

`Assignment2_Report.pdf`

7. Screenshots

Include execution and output screenshots

40. One important improvement to the assignment compare the dataset before and after cleaning.

For example:

Quality Measure	Before Cleaning	After Cleaning
Total records	XXXXX	XXXXX
Missing values	XXXXX	0/XXXXX
Duplicate records	XXXXX	0
Invalid records	XXXXX	0
Outliers	XXXXX	Reduced/removed
Final records	XXXXX	XXXXX

This makes the practical much more meaningful because you can **demonstrate the effect of preprocessing**, rather than merely executing Spark commands.
Recommended practical sequence

Dataset → Spark → Exploration → Missing Values → Duplicates → Invalid Data → Outliers → Noise → Validation → Train/Test Split → Parquet Storage

This gives you a complete **Big Data practical rather than a basic Python data-cleaning exercise**.

https://chatgpt.com/c/6a992318-5790-83e9-854f-1acdb943d51a